



Шаг пройден

[🏠](#) / [Мое](#) / [Мои проекты](#) / AI-агент службы поддержки

Проект: AI-агент службы поддержки

15 августа
2026 г.

> НЕДОСТ...

📄 НЕДОСТ...

○ В ПРОЦ...

[🔗 Репозиторий](#)[🔍 Помощь](#)

Подготовка

✓ 1

✓ 2

✓ 3

✓ 4

✓ 5

> 6

> 7

> 8

> 9

> 10

📄 Проверка

Углубление workflow: авто-эскалация тикетов в YDB

Превращаем простой «агент отвечает на обращение» в осмысленную цепочку по расписанию: раз в день находим открытые тикеты, которые давно ждут ответа, → агент формирует дайджест для оператора → дайджест отправляется оператору по почте. Заодно осваиваем ретрай, обработку ошибок и трассировку шагов.

Pull-режим workflow (по расписанию — раз в день или раз в час для демо). Источник данных — YDB из шага про MCP и YDB:

```
1  Timer (cron 09:00)
2  |
3  ▼
4  Workflow
5  ├── databaseQuery → открытые тикеты старше 24 часов (YQL SELECT)
6  ├── switch       → если просроченных нет, выйти в терминальный шаг
7  ├── aiStudioAgent → дайджест по выборке для оператора
8  ├── databaseQuery → пометить выбранные тикеты как escalated (YQL UPDATE)
9  └── httpCall (POST) → отправить дайджест оператору по почте (через CF email-sender)
```



Шаги с внешними вызовами (`databaseQuery` , `aiStudioAgent` , `httpCall`) стоит снабдить `retryPolicy` на `5xx/timeout` — тогда разовая ошибка YDB или AI Studio не пороняет workflow.



Все курсы

- [Шаг AIStudioAgent в Yandex workflows](#) — инструкция от Яндекса
- [Шаг DatabaseQuery \(SQL-запросы к YDB\) в Yandex Workflows](#) — инструкция от Яндекса
- [Формат cron-выражения](#) — таблица шести полей и правило про ?
- [JSON-схема YaWL](#) — машиночитаемая спецификация: полный список полей каждого шага, коды ошибок для `retryPolicy`, границы значений. Ей же можно проверить свой `workflow.yaml` до деплоя

Задачи

1. Опишите `src/workflow.yaml` в формате YaWL 0.1. Это не `--file workflow.yaml` и не `deploy-revision` — используется `yc serverless workflow create --yaml-spec`. Шаги в YaWL имеют camelCase-имена (`databaseQuery`, `aiStudioAgent`, `httpCall`, `switch`, `success`) и `next` живёт внутри шага, а не снаружи. Шаблоны — jq-интерполяция `'\(...)'`, не `${...}` (последнее даёт невнятное `Internal error`).

- `databaseQuery` — SQL-запрос к YDB/Postgres/MySQL/ClickHouse. Документация: <https://yandex.cloud/ru/docs/serverless-integrations/concepts/workflows/yawl/integration/databasequery>.
- `aiStudioAgent` — вызов агента из Agent Atelier (тот же `agent_id`, что на шаге про настройку AI Studio). Документация: <https://yandex.cloud/ru/docs/serverless-integrations/concepts/workflows/yawl/integration/aistudioagent>.

2. Граф шагов соберите сами. Требуемая последовательность:

- `databaseQuery` — выбрать открытые тикеты старше порога (24 часа);
- `switch` — если просроченных нет, уйти в терминальный шаг (`y switch нет default` — добавьте `catch-all condition: "true"`);
- `aiStudioAgent` — сформировать дайджест по выборке (тот же `agent_id`, что на шаге про настройку AI Studio);
- `databaseQuery` — пометить выбранные тикеты как `escalated` (YQL UPDATE с тем же условием, что и SELECT), чтобы завтра они не попали в дайджест повторно;
- `httpCall` — отправить дайджест оператору по почте через CF `email-sender` (в Python-примерах — `src/email_sender.py`); YaWL напрямую SMTP не умеет, поэтому HTTP-вызов на функцию-обёртку. Адрес оператора функция берёт из своего окружения (`OPERATOR_EMAIL`), а не из тела запроса — почему так, в «Подсказках».

Форма YaWL (dummy-пример — показывает только синтаксис, не бизнес-логику):

```
1  yawl: "0.1"
2  start: step_one
3  steps:
```





Все курсы

```

7         type: YDB
8         host: ydb.serverless.yandexcloud.net
9         port: 2135
10        database: /ru-central1/<cloud-id>/<db-id>
11        ssl: true
12        iam: true                # workflow ходит в YDB от своего SA
13        query: "SELECT 1"        # реальный SQL — ваш
14        mode: QUERY
15        next: check
16    check:
17        switch:
18        choices:
19            - condition: "true"    # catch-all, поля `default` НЕТ
20              next: done
21    done:
22    success: {}

```

3. Назначьте SA workflow'a роли: `ydb.editor` , `ai.assistants.editor` , `ai.languageModels.user` .

Без `ai.assistants.editor` шаг `aiStudioAgent` падает с 403 Forbidden на

`https://ai.api.cloud.yandex.net/v1/responses` . Команда — по аналогии с шагом про настройку AI Studio (одна роль за вызов):

```

1  for ROLE in ydb.editor ai.assistants.editor ai.languageModels.user; do
2      yc resource-manager folder add-access-binding \
3          --id <folder-id> --service-account-id <SA_ID> --role "$ROLE"
4  done

```



4. Создайте workflow:

```

1  yc serverless workflow create \
2      --name daily-escalation \
3      --yaml-spec src/workflow.yaml \
4      --service-account-id <SA_ID>

```



Обновление через `yc serverless workflow update --yaml-spec ...` .

5. Запланируйте расписание (cron из 6 полей `min hour dom mon dow year` ; `timezone` отдельно). Поля `dom` и `dow` нельзя заполнить одновременно — в одном из них ставится `?` , поэтому «каждый день в 9:00» записывается со знаком вопроса в дне недели:



Все курсы

```

3  --yaml-spec src/workflow.yaml \
4  --schedule-cron-expression "0 9 * * ? *" \
5  --schedule-timezone Europe/Moscow

```

Важно: после назначения расписания `scheduled-trigger` запускает workflow от имени своего SA. Если SA триггера совпадает с SA workflow'a (как у нас — `ai-studio-sa`), всё равно нужны две роли на workflow. Без них каскад ошибок:

- **НЕТ** `serverless.workflows.executor` → service account does not have rights to start the workflow (или `400 can't invoke workflow`)
- **НЕТ** `serverless.workflows.viewer` → `400 doesn't have get permission for workflow` (API делает GET перед запуском, поэтому viewer нужен даже для invoke)

Рабочий способ — выдать обе роли SA workflow'a:

```

1  yc serverless workflow add-access-binding \
2  --name daily-escalation \
3  --role serverless.workflows.executor \
4  --service-account-id <SA_ID>
5  yc serverless workflow add-access-binding \
6  --name daily-escalation \
7  --role serverless.workflows.viewer \
8  --service-account-id <SA_ID>

```



Проверка: `yc serverless workflow list-access-bindings --name daily-escalation` — должна показать обе строки.

`yc serverless workflow allow-unauthenticated-execution` формально существует, но в текущем YC не сохраняется (флаг `executionPermissions` остаётся пустым) — поэтому не полагайтесь на него.

6. Ручной тест (до расписания): поставьте порог в `Interval('PT1H')` вместо `'PT24H'`, чтобы существующие тикеты попали в выборку; запустите `yc serverless workflow execution start daily-escalation`; проверьте `execution get` и приход письма оператору. Потом верните `'PT24H'`.
7. Умышленно сломайте секрет/роль → проверьте, что шаг падает с понятной ошибкой (например `STEP_PERMISSION_DENIED` если забыть роль), в `execution get` видны и `error_code`, и сообщение.

Подсказки

- В YDB SQL нет `CurrentTimestamp()` — используйте `CurrentUtcTimestamp()`.



терминальные шаги — `success: {}` и `fail: { errorMessage: "..."} .`

- Дайджест оператору шлёт через CF `email-sender: httpCall` с `method: POST` и JSON-телом `{subject, body}`. YaWL сырой SMTP не умеет — функция-обёртка отправляет письмо по SMTP за workflow (в Python-варианте — `smtpplib.SMTP_SSL`).
- Адресата функция берёт из `OPERATOR_EMAIL`, а не из запроса, и это не придирка к стилю. `httpCall` не шлёт IAM-токен, поэтому функцию приходится открывать без аутентификации — значит дёрнуть её может любой, кто узнал URL. Пока адресат приходит в теле, это рассылка с корпоративного ящика на любой адрес; с адресатом из окружения худшее, что можно получить, — лишний дайджест оператору.
- Ретраи имеют смысл только на `5xx/timeout: STEP_PERMISSION_DENIED` ими не лечится — это не временная ошибка, и повтор лишь оттянет понятный отказ по правам.
- Форма политики: `retryPolicy` — поле внутри тела шага, рядом с `next` и `timeout`; `defaultRetryPolicy` на уровне процесса задаёт её сразу всем шагам. Обязательное поле одно — `errorList`, коды берутся из перечисления в схеме (`STEP_TIMEOUT`, `HTTP_CALL_503`, `DATABASE_QUERY_UNAVAILABLE` и так далее), режим выбора задаёт `errorListMode: INCLUDE` или `EXCLUDE`. Задержки пишутся **только в секундах** (`initialDelay: 2s`, `maxDelay: 30s`) — `1m` и `1h` схема отклонит; `retryCount` не больше 100.

Результат шага

- Workflow отработывает по расписанию; оператору на `OPERATOR_EMAIL` приходит письмо-дайджест по просроченным тикетам.
- В YDB обработанные тикеты переходят в статус `escalated` (видно через `SELECT status, count(*) FROM tickets GROUP BY status`) — повторно в дайджест они не попадают.
- В трейсах YC Logging видно каждый шаг отдельно (вход, выход, длительность, токены).
- Вы можете объяснить, чем pull-режим poller'a (CF по таймеру, шаг про email-workflow) отличается от workflow по расписанию (этот шаг) и когда какой подход применять.

Завершить шаг

